5.3 Modelo de clases

Proyecto: Monitor de Consumo Eléctrico — EnergyMonitor **Ficha:** 3145556 — Análisis y Desarrollo de Software (ADSO), SENA CIES Regional Huila **Versión:** 1.0

> **Nota de actualización (27/10/2026).** Este documento describe el diseño de la versión 1.0, cuando el backend era el monolito modular `energy-monitor-backend` (Spring Modulith). Ese diseño evolucionó: los siete contextos acotados son hoy **siete microservicios** detrás del API gateway `en-monitor-api-gateway` (8080) — seguridad 8082, dispositivos 8084, mediciones 8085, hogares 8086, notificaciones 8087, alertas 8088 y recomendaciones 8089 —, comunicados por REST interno (`/internal/**`) y eventos de RabbitMQ con outbox transaccional, sobre el mismo esquema MySQL compartido (37 tablas). El backend modular quedó retirado. Las capas hexagonales, los contextos y los patrones descritos aquí se conservan dentro de cada servicio; la arquitectura vigente, las rutas, los eventos y la configuración están en el manual técnico (secciones 9 a 13) y el modelo de datos vigente en `06-base-datos`.

1. Propósito

El modelo de clases describe la estructura interna del sistema: las clases, las interfaces, las enumeraciones y las relaciones que componen el dominio. A diferencia del modelo de datos, que responde a «cómo se almacenan los datos», el modelo de clases responde a «cómo se comporta el sistema»: qué responsabilidades tiene cada clase, qué métodos expone y cómo colaboran entre sí.

Este modelo es la base conceptual de los patrones de diseño descritos en `05.4-patrones-de-diseno.md`.

2. Diagrama de clases

El diagrama base se encuentra en `assets/diagramas/Diagrama_clases.wsd` (sintaxis PlantUML, renderizable en https://www.plantuml.com/plantuml/uml).

Figura 2. *Diagrama de clases del dominio, organizado en paquetes: Strategy, enumeraciones, dominios y patrones Observer / Factory Method.*

> **Nota de versión.** El diagrama adjunto fue elaborado en una fase anterior del proyecto. Varias clases que allí se marcan como «pendiente de implementación» ya existen hoy en el código, y las enumeraciones han crecido. La sección 10 documenta la diferencia con precisión.

3. Paquetes del modelo

Paquete	Contenido	Responsabilidad
`Strategy_Pattern`	`UserPermissions`, `MemberPermissions`, `OwnerPermissions`	Autorización según el rol dentro del hogar
`enums`	Enumeraciones de dominio y de API	Conjuntos de valores permitidos
`Domain_Security`	`Person`, `User`, `UserConfiguration`, `PasswordResetToken`, `UserSession`, `RefreshToken`, `SecurityConfiguration`, `PasswordPolicy`, `LoginErrorLog`, `Email`, `PasswordHash`	Identidad, credenciales y sesiones
`Domain_Access_Control`	`SystemRole`, `UserSystemRole`, `Permission`, `SystemRolePermission`, `AuditLog`, `CheckPermission`	RBAC y auditoría
`Domain_Home_Management`	`HomeType`, `HomeThresholds`, `Home`, `UserHome`, `Role`	Hogares, miembros y umbrales

Paquete	Contenido	Responsabilidad
`Domain_Devices`	`ApplianceType`, `Device`, `DeviceStatusLog`, `DeviceHome`, `DeviceStatus`	Dispositivos de medición
`Domain_Measurements`	`Measurement`, `ConsumptionLevel`	Mediciones y niveles de riesgo
`Domain_Notifications`	`Alert`, `Recommendation`, `Notification`, `NotificationPreference`, `PushSubscription`	Eventos, entregas y preferencias
`Observer_Pattern`	`MeasurementObserver`, `AlertObserver`, `RecommendationObserver`	Reacción ante mediciones
`Factory_Method_Pattern`	`AlertFactory`, `RecommendationFactory`, `NotificationFactory`	Construcción de eventos y avisos
`Recommendation`	`RecommendationRules`, `Suggestion`, `MonitoredDevice`, `HourSample`, `ConsumptionLimit`	Servicio de dominio del análisis de consumo

4. Notación UML utilizada

Símbolo	Significado
`+`	Método público
`-`	Atributo privado
`{abstract}`	Clase o método abstracto
`<<interface>>`	Interfaz
`<<FK>>`	Atributo que referencia otra entidad, sin asociación navegable
`..>`	Dependencia (uso)
`--`	Asociación
`*--`	Composición (el todo controla el ciclo de vida de la parte)
`o--`	Agregación

> **Convención de este proyecto:** las relaciones «dibujadas» hacia otras tablas de contexto se representan como **dependencia punteada** (`..>`), no como asociación. Es la consecuencia gráfica de la decisión de mapeo relacional plano descrita en `05.2-modelo-de-datos.md`, sección 4.4.

5. Clases del dominio de seguridad

5.1 `Person`

Persona humana asociada a una cuenta. Deliberadamente **no** almacena datos de contacto ni dirección.

Atributo	Tipo	Visibilidad
`idPerson`	`String`	`-`
`name`	`String`	`-`
`lastName`	`String`	`-`

Operación	Responsabilidad
`+ rename(name, lastName)`	Cambia nombre y apellido validando que no queden vacíos

Decisiones de diseño documentadas en el modelo y en el código:

Atributo descartado	Motivo	Cambioset que lo eliminó
`person.address`	La dirección autoritativa es `home.address`; duplicarla generaría inconsistencias	`security-018`
`person.cellphone`	La cuenta se identifica exclusivamente por correo electrónico	`security-020`
`person.profileImage`	El avatar pertenece a la cuenta, no a la persona; vive en `User`	`security-017`

5.2 `User`

Cuenta de acceso. Concentra credenciales, estado y trazabilidad de la sesión.

Atributo	Tipo	Visibilidad
`idUser`	`String`	`-`
`personId`	`String`	`-`
`passwordHash`	`PasswordHash`	`-`
`email`	`Email`	`-`
`emailVerified`	`boolean`	`-`
`registrationDate`	`Instant`	`-`
`status`	`UserStatus`	`-`
`failedLoginAttempts`	`int`	`-`
`lastLoginAt`	`Instant`	`-`
`profileImage`	`String`	`-`

Operación	Responsabilidad
`+ canAuthenticate()`	Indica si la cuenta está `ACTIVE` y el correo verificado
`+ changePassword(newHash)`	Sustituye el hash y revoca las sesiones activas
`+ changeEmail(newEmail)`	Cambia el correo y obliga a volver a verificar
`+ changeProfileImage(profileImage)`	Actualiza la imagen (data URL)
`+ verifyEmail()`	Marca el correo como verificado
`+ activate()` / `+ deactivate()` / `+ block()`	Transiciones de estado
`+ incrementFailedLoginAttempts()`	Contabiliza intentos fallidos
`+ resetFailedLoginAttempts()`	Restablece el contador tras un inicio exitoso
`+ recordSuccessfulLogin(instant)`	Registra `lastLoginAt`

> **Patrón aplicado — Objeto de dominio (Entity).** El comportamiento de `User` no se delega a un servicio externo: las reglas de transición de estado (`ACTIVE → BLOCKED`) y el control de invariantes viven dentro de la clase. Esto mantiene la invariante del agregado garantizada sin depender de que cada llamador recuerde validarla.

5.3 Objetos de valor: `Email` y `PasswordHash`

Ambos son **records inmutables** con factoría estática de validación (`Email.of(...)`, `PasswordHash.of(...)`) y accesor sin prefijo `get` (`user.email()`).

Propiedad	Consecuencia de diseño
Invariantes en la construcción	Una instancia de `Email` siempre contiene un correo con formato válido
Igualdad por valor	Dos `Email` con el mismo contenido son iguales; el `Set` de correos funciona
Sin identidad	No son entidades; no tienen llave primaria ni persistencia propia

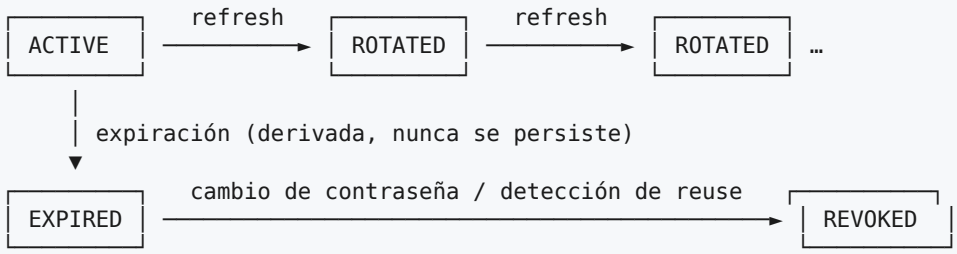
> **Patrón aplicado — Objeto de valor.** Es la pieza que evita que el dominio manipule cadenas sueltas para correos y contraseñas, y que concentra en un solo lugar las reglas de formato.

5.4 `UserSession` y `RefreshToken`

El par de clases que implementa el ciclo de vida de la sesión con rotación de tokens.

Clase	Operaciones clave
`UserSession`	`rotate(newExpiration, rotatedAt)`, `revoke(revokedAt, reason)`, `close(instant)`, `isExpired(now)`, `isActive(now)`
`RefreshToken`	`RefreshToken.root(...)`, `RefreshToken.childOf(...)`, `isExpired(now)`, `isUsable(now)`, `isReplayed()`, `rotate(rotatedAt)`, `revoke(revokedAt, reason)`

Máquina de estados de `RefreshTokenStatus`:



Composición `UserSession 1 *-- 1..* RefreshToken`. Una sesión posee todas las generaciones de refresh token generadas bajo ella. Al revocar la sesión, todas quedan invalidadas: es la implementación del cierre de sesión global.

Auto-relación `RefreshToken 0..1 -- 1 RefreshToken : parent`. Una generación apunta a la que reemplaza, formando una cadena genealógica que permite detectar la reutilización de un token ya rotado.

Deuda técnica declarada en el código: la familia de tokens no tiene límite de tamaño; un atacante que Robbarde un `refresh_token` válido puede acumular generaciones hasta que se detecte la anomalía.

5.5 `PasswordPolicy` y `CheckPermission`

Clase	Operación	Regla
`PasswordPolicy`	`+ validate(password)`	Longitud 8–64, exige mayúsculas, números y símbolos, vigencia de 90 días
`CheckPermission`	`+ check(idUser, permissionCode)`	Recorre la cadena RBAC de cinco tablas contra el estado vivo

> `CheckPermission` es la **única vía de autorización real** del sistema. No contiene lógica por tipo de rol ni interpreta permisos embebidos en el JWT: resuelve siempre contra la base de datos, de modo que un cambio de rol surte efecto inmediato sin esperar la expiración del token.

6. Clases del dominio de hogares

Clase	Operaciones
`HomeType`	Catálogo de tipos de hogar
`HomeThresholds`	`+ resetToDefaults()`
`Home`	`+ addDevice(d)`, `+ removeDevice(d)`, `+ addUser(u, role)`, `+ removeUser(u)`, `+ updateThresholds(daily, monthly)`, `+ getAllDevices()`

Clase	Operaciones
`UserHome`	`+ toggleFavorite()`, `+ leaveHome()`, `+ getPermissions()`

Decisiones de diseño:

Decisión	Justificación
`Home 1 *-- 1 HomeThresholds` (composición)	Los umbrales no tienen existencia propia sin un hogar. Si se elimina el hogar, sus umbrales se eliminan con él.
`Home "*" o-- "1" HomeType` (agregación)	El tipo de hogar es un catálogo compartido: un mismo `HomeType` sirve a muchos hogares y no «pertenece» a ninguno.
`Home 1 -- "0..*" UserHome` (asociación)	La pertenencia al hogar existe por sí misma, incluso si el hogar se elimina.
`+ getPermissions()` en `UserHome`	Devuelve el objeto de permisos (`UserPermissions`) correspondiente al rol. Punto de extensión del patrón Strategy.

7. Clases del dominio de dispositivos y mediciones

Clase	Operaciones
`Device`	`Device.create(...)`, `Device.relinked(...)`, `Device.edited(...)`, `+ withApiKey(newKey)`, `+ takeMeasurement()`
`DeviceStatusLog`	`DeviceStatusLog.create(...)` — estado, intensidad de señal y última conexión
`Measurement`	`+ getVoltage()`, `+ getCurrent()`, `+ getActivePower()`, `+ getStoredEnergy()`
`ConsumptionLevel`	Rango `minLimit – maxLimit` del nivel de riesgo

> **Patrón aplicado — Sujeto / Observador.** El diagrama de clases modela `MeasurementObserver`, `AlertObserver` y `RecommendationObserver` como observadores de `Device`. En la implementación, el papel de sujeto lo ejerce el **publicador de eventos** del módulo `measurement`, y el papel de observador lo desempeña el oyente `MeasurementRecordedListener` del módulo `alert`. El contrato se conserva; el transporte cambia de invocación directa a evento en proceso (ver `05.4-patrones-de-diseno.md`, sección 4).
>
> **Patrón aplicado — Objeto de valor.** `Measurement` no tiene identidad propia más allá de `idMeasurement` y expone sus magnitudes mediante getters. Las magnitudes eléctricas son valores inmutables: no pueden cambiar una vez registrada la medición, lo que garantiza la confiabilidad del histórico.

8. Clases de eventos y avisos

Clase	Operaciones
`Alert`	Fábricas `Alert.threshold(...)`, `Alert.limit(...)`, `Alert.connectivity(...)`, `Alert.device(...)`; `+ resolveAlert()`
`Recommendation`	`+ markAsRead()`, `+ markAsUnread()`
`Notification`	`+ settled(...)` (`SENT`/`FAILED`), `+ read(...)`
`NotificationPreference`	`NotificationPreference.defaults()`
`RecommendationRules`	`+ evaluate(instant, zoneId, devices, samples, limit)`

Decisión de diseño central: `Notification` no es la clase base de `Alert` y `Recommendation`.

En versiones previas del modelo era habitual modelar `Notification` como superclase de los eventos. El modelo definitivo las separa porque responden a preguntas distintas:

Clase	Pregunta que responde	Contexto
`Alert` / `Recommendation`	**¿Qué ocurrió?**	Alerts / Recommendations
`Notification`	**¿El mensaje llegó, por qué canal y si fue leído?**	Notifications

`Notification` referencia el evento origen mediante el par `sourceType` + `sourceId`, sin acoplarse a su esquema. Además, sus ciclos de estado (`deliveryStatus` y `readAt`) son **independientes entre sí e independientes de** `alert.alertStatus`:

> Una alerta entregada no es una alerta resuelta.

Clase inmutable por excepción. `RecommendationEntity` es la **única** entidad de persistencia del proyecto sin setters: declara únicamente `@PrePersist` y `@PreUpdate` propios, y solo permite cambiar el estado de lectura. Esto refuerza la invariante de que las recomendaciones son inmutables una vez creadas.

8.1 `RecommendationRules` — servicio de dominio

Es la **única** clase de `domain/service` del proyecto. Es puramente funcional: no recibe dependencias, no realiza entrada/salida y no lee el reloj (el instante y la zona horaria se pasan como parámetros).

Decisión de diseño	Justificación
Sin dependencias	El mismo conjunto de reglas puede probarse exhaustivamente sin contenedor de Spring
Reloj y zona horaria como entrada	El resultado es determinista y reproducible; facilita las pruebas de reglas de consumo
Vive en el dominio, no en la aplicación	Las reglas de negocio no dependen del framework

> **Patrón aplicado — Servicio de dominio.** Cuando un comportamiento atraviesa varios agregados y no pertenece naturalmente a ninguno, se extrae a un servicio de dominio. Las reglas de recomendación analizan mediciones de **todos** los dispositivos de un hogar, por lo que no corresponde a `Measurement`, a `Device` ni a `Recommendation` en particular.

9. Enumeraciones del dominio

Enumeración	Valores	Ubicación	Uso
`UserStatus`	`ACTIVE`, `INACTIVE`, `BLOCKED`	Dominio	Estado de la cuenta
`AuditAction`	`CREATE`, `UPDATE`, `DELETE`, `LOGIN`, `LOGIN_FAILED`, `LOGOUT`	Dominio	Acción auditada
`LoginErrorType`	`INVALID_PASSWORD`, `USER_NOT_FOUND`, `ACCOUNT_BLOCKED`, `ACCOUNT_INACTIVE`	Dominio	Clasificación del fallo de acceso
`RefreshTokenStatus`	`ACTIVE`, `ROTATED`, `REVOKED`, `EXPIRED`	Dominio	Ciclo del token (`EXPIRED` es derivado, no se persiste)
`RevocationReason`	`REFRESH_TOKEN_REUSE`, `PASSWORD_CHANGED`, `ADMINISTRATIVE`	Dominio	Motivo de revocación
`Role`	`OWNER`, `MEMBER`	Dominio	Rol del usuario dentro del hogar
`LimitPeriod`	`DAILY`, `MONTHLY`	API	Periodo del umbral en edición
`DeviceStatus`	`ONLINE`, `OFFLINE`	Dominio	Conectividad del dispositivo
`ConnectivityStatus`	`ONLINE`, `OFFLINE`	API	Conectividad expuesta al cliente
`RiskConsumption`	`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`	API	Nivel de riesgo del consumo
`AlertType`	`THRESHOLD`, `LIMIT`, `CONNECTIVITY`, `DEVICE`	API	Origen de la alerta
`AlertStatus`	`PENDING`, `RESOLVED`	API	Ciclo de vida de la alerta

Enumeración	Valores	Ubicación	Uso
`RecommendationType`	`SAVING`, `PEAK_HOURS`, `HIGH_CONSUMPTION`, `HISTORICAL_COMPARISON`, `THRESHOLD`	API	Motivo de la recomendación
`RecommendationStatus`	`READ`, `UNREAD`	API	Lectura de la recomendación
`NotificationStatus`	`PENDING`, `SENT`, `FAILED`	Dominio	Resultado del envío
`NotificationChannel`	`EMAIL`, `PUSH`	Dominio	Canal de entrega
`NotificationSourceType`	`ALERT`, `RECOMMENDATION`, `SECURITY`	Dominio	Contexto que originó el mensaje
`Access` (en `AuthorizeBrokerAccess`)	`READ`, `WRITE`, `READ+WRITE`, `SUBSCRIBE`	Dominio	Permisos de acceso al bróker MQTT

> **Todas las enumeraciones están declaradas en Java**, a diferencia de las columnas `ENUM` de MySQL que comparten nombre. Esto permite que el dominio y la base de datos compartan vocabulario sin acoplar la lógica de negocio al motor de almacenamiento. Las de la columna «API» viven en el paquete `api/` del módulo porque forman parte del contrato público.

10. Diferencias entre el artefacto de diseño y el modelo implementado

#	Aspecto	Diagrama del artefacto	Código vigente	Origen del cambio
1	`UserPermissions`, `MemberPermissions`, `OwnerPermissions`	Marcadas como *«pendiente de implementación»*	**Implementadas** en `home/domain/model`; `UserHome.getPermissions()` devuelve la estrategia correspondiente	Requisito RF-HOM-05 (gestión de permisos)
2	`RecommendationAnalyzer`	Clase con `analyze()`, `detectPatterns()`, `generateRecommendation()`	Sustituida por `RecommendationRules` (servicio de dominio) + `EvaluateRecommendationsService` (caso de uso) + `RecommendationFactory`	Separación entre regla de negocio y caso de uso
3	`AlertType`	`THRESHOLD`, `CONNECTIVITY`	+ `LIMIT`, `DEVICE`	Alertas de límite acumulado y de vinculación
4	`Notification` sin subclases	Sin `NotificationPreference` ni `PushSubscription`	Ambas clases agregadas a `Domain_Notifications`	Preferencias de canal y Web Push
5	`Device.subscribe/unsubscribe/notify`	Métodos del sujeto	Sustituidos por publicación y escucha de eventos Spring (`ApplicationEventPublisher` + `@EventListener`)	Desacoplamiento entre contextos
6	`Notification.markAsSent/markAsFailed/markAsRead`	Tres métodos	Una sola operación `settled(...)` más `read(...)`	Simplificación de la transición de estados
7	`SystemConfig`	Mencionada en comentario sobre `useSystemDefault`	**Eliminada**; el valor por defecto vive en `HomeProperties` (configuración, no base de datos)	Cierre del hueco de diseño
8	`Recommendation` como evento de su contexto	Idéntico	Se mantiene; `status` se materializa en `NotificationsStatus`	Sin cambios

Recomendación: actualizar `Diagrama_clases.wsd` con estas ocho diferencias antes de la entrega final, en particular la primera, que hoy hace creer que la autorización por rol no existe.

11. Patrones de diseño identificados en el modelo de clases

Paquete del diagrama	Estado	Implementación concreta
`Strategy_Pattern`	**Implementado**	`UserPermissions` con dos estrategias concretas por rol
`Observer_Pattern`	**Implementado con transporte de eventos**	`MeasurementRecorded` publicado por `measurement` y escuchado por `alert`
`Factory_Method_Pattern`	**Implementado**	Fábricas estáticas en los agregados: `Alert.threshold()`, `RefreshToken.root()`, `Device.create()`
Objeto de valor	**Implementado**	`Email`, `PasswordHash`, `Measurement`
Objeto de dominio	**Implementado**	Comportamiento en `User`, `RefreshToken`, `UserSession`
Servicio de dominio	**Implementado**	`RecommendationRules`
Singleton	**Implementado por el contenedor**	Beans de Spring, `Clock`, `ThreadPoolExecutor`

El detalle completo de cada patrón, con la comparación entre el patrón canónico y su aplicación real, se desarrolla en `05.4-patrones-de-diseno.md`.

12. Matriz de trazabilidad clase → requisito

Clase o paquete	Requisito (especificación de requisitos)
`Person`, `User`, `UserConfiguration`	RF-SEC-01, RF-SEC-08
`Email`, `PasswordHash`, `PasswordPolicy`	RF-SEC-12
`PasswordResetToken`, `LoginErrorLog`	RF-SEC-06; RNF-SEG-05, RNF-SEG-06
`UserSession`, `RefreshToken`	RF-SEC-04, RNF-SEG-01, RNF-SEG-02 (sesión persistente)
`SystemRole`, `UserSystemRole`, `Permission`, `SystemRolePermission`, `CheckPermission`	RF-SEC-09 (gestión de permisos)
`AuditLog`	RF-SEC-12; RNF-SEG (seguridad de la información)
`Home`, `HomeType`, `UserHome`	RF-HOM-01 a RF-HOM-04
`HomeThresholds`	RF-HOM-08
`UserPermissions`, `MemberPermissions`, `OwnerPermissions`	RF-HOM-05 (autorización por rol)
`Device`, `ApplianceType`, `DeviceStatusLog`	RF-DEV-02 a RF-DEV-06; RF-MED-04 a RF-MED-06
`Measurement`, `ConsumptionLevel`	RF-MED-01, RF-MED-02, RF-MED-09
`Alert`	RF-ALE-01 a RF-ALE-06
`Recommendation`, `RecommendationRules`	RF-REC-01 a RF-REC-09
`Notification`, `NotificationPreference`, `PushSubscription`	RF-NOT-01, RF-NOT-02, RF-NOT-04 (canal de entrega); RF-NOT-03

13. Artefactos relacionados

Artefacto	Ubicación
Diagrama de clases (PlantUML)	`assets/diagramas/Diagrama_clases.wsd`
Modelo de datos	`05.2-modelo-de-datos.md`
Patrones de diseño	`05.4-patrones-de-diseno.md`